



Há **62 anos**
reescrevendo histórias!

The background of the slide is a dark, blue-toned image showing a laptop and two smartphones. The laptop screen displays lines of code in a dark theme. One smartphone shows a mobile app interface with a blue header and white content. The other smartphone shows a mobile app interface with a blue header and white content, including a shopping cart icon and a gear icon. Above the devices, there are circular icons representing various technologies: a blue arrow, a green Android robot, a white Apple logo, a white cloud with a database cylinder, and a white gear. The text is overlaid on this background.

Sistemas de Informação

Desenvolvimento de Software para Mobile.

Android: Web Hooks e API's.

Prof. Tiago D. F. Cassimiro
tiago.cassimiro@unifeb.edu.br
2026/2

Centro Universitário da Fundação Educacional de Barretos



Há **62 anos**
reescrevendo histórias!

Android: Webhooks e APIs para Conectividade

Dominando a comunicação assíncrona e em tempo real em aplicações móveis.



Há **62 anos**
reescrevendo histórias!

Sumário

- Ecossistema de Conectividade Android: APIs e JSON
- Arquitetura REST, Métodos HTTP e Status
- Retrofit e OkHttp: Implementação Prática
- Webhooks: Conceito e Sincronização em Tempo Real
- Segurança e Melhores Práticas em Conectividade



Há **62 anos**
reescrevendo histórias!

Atualmente, grande parte dos aplicativos móveis depende de serviços externos.

Por exemplo, um aplicativo de:

banco;

delivery;

comércio eletrônico;

redes sociais;

previsão do tempo;

streaming;

transporte;

educação.



Há **62 anos**
reescrevendo histórias!

O aplicativo Android normalmente funciona como cliente, enquanto os dados e regras de negócio ficam em um servidor.

Uma arquitetura simplificada é:





Há **62 anos**
reescrevendo histórias!

O que é uma API?

Application Programming Interface

Em português:

Interface de Programação de Aplicações.

Uma API define uma forma padronizada para que diferentes sistemas possam se comunicar.

Exemplo

Um aplicativo Android de uma loja.

O aplicativo precisa consultar os produtos disponíveis.

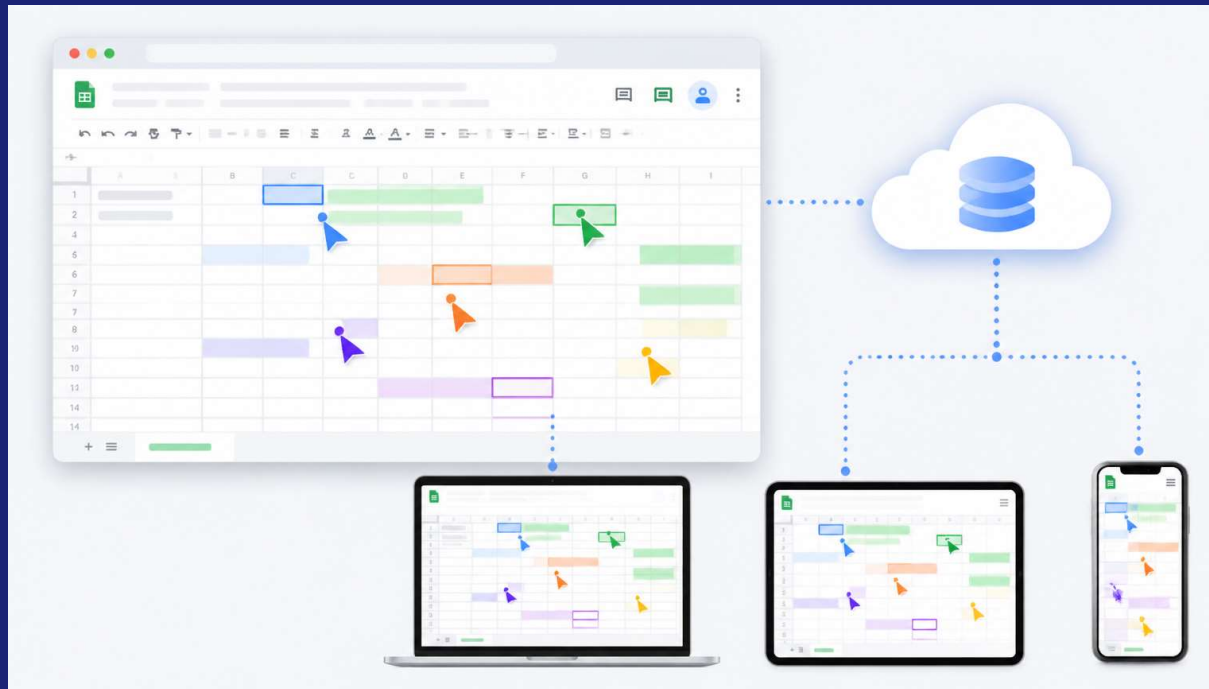
Ele poderia fazer:

GET <https://api.loja.com/produtos>

O servidor poderia responder:

```
[  
  {  
    "id": 1,  
    "nome": "Notebook",  
    "preco": 3500.00  
  },  
]
```

Ecossistema de Conectividade Android



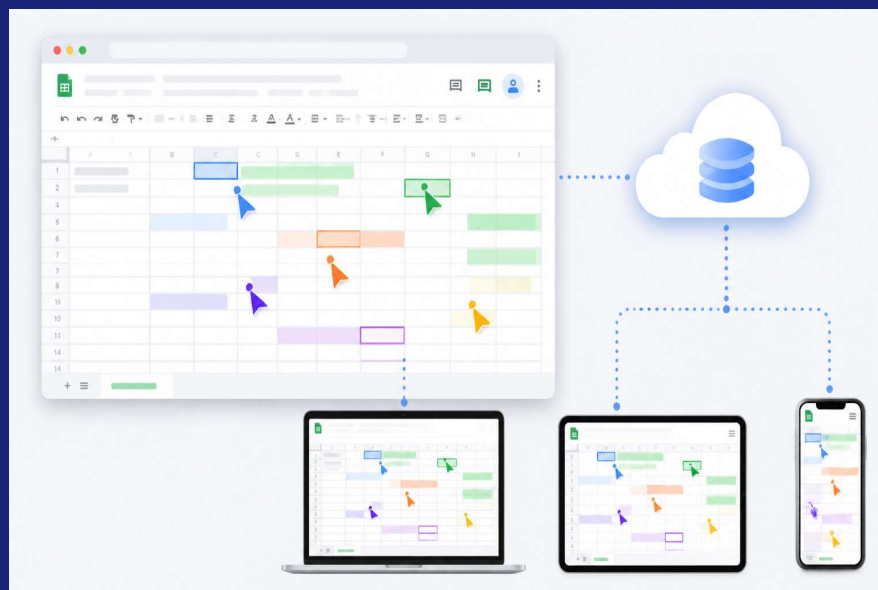
Colaboração em tempo real com sincronização de planilhas na nuvem.

A conectividade é o pilar das aplicações modernas. No Android, a comunicação com servidores externos ocorre majoritariamente através de **APIs RESTful** (*Interfaces de Programação de Aplicações*), que permitem ao app solicitar dados ou executar operações em um servidor remoto..



Há **62 anos**
reescrevendo histórias!

Ecosystema de Conectividade Android



Colaboração em tempo real com sincronização de planilhas na nuvem.

O *JavaScript Object Notation (JSON)* é o padrão de fato para intercâmbio de dados devido à sua leveza e facilidade de *parsing*. No entanto, desafios como latência, perda de pacotes e consumo de bateria exigem implementações eficientes e assíncronas.



Há **62 anos**
reescrevendo histórias!

Fundamentos da Arquitetura REST

Cliente-Servidor & Stateless

Separação de preocupações: o app Android cuida da UI e o servidor do processamento. Cada requisição deve conter todas as informações necessárias para ser processada sem depender de estado no servidor.



Há **62 anos**
reescrevendo histórias!

Fundamentos da Arquitetura REST

Interface Uniforme

Uso padronizado de recursos identificados por URIs (Uniform Resource Identifiers), garantindo que a comunicação entre o Android e o backend siga contratos previsíveis.



Há **62 anos**
reescrevendo histórias!

Fundamentos da Arquitetura REST

Cacheable

Respostas devem definir explicitamente se podem ou não ser armazenadas localmente para economizar banda e melhorar a performance em dispositivos móveis.

Métodos e Status HTTP

A comunicação Android-Servidor utiliza verbos HTTP para definir a ação desejada e códigos de status para interpretar o resultado.

Verbo	Ação	Descrição
GET	Leitura	Recupera dados de um recurso.
POST	Escrita	Cria um novo recurso no servidor.
PUT	Atualização	Substitui um recurso existente integralmente.
DELETE	Remoção	Remove um recurso específico.
PATCH	Modificação	Atualiza parcialmente um recurso.



Há **62 anos**
reescrevendo histórias!

O que é JSON ?

O JSON — JavaScript Object Notation é um dos formatos mais utilizados para transportar informações entre aplicativos e APIs.

Exemplo:

```
{  
  "id": 10,  
  "nome": "João",  
  "curso": "Sistemas de Informação",  
  "idade": 21  
}
```



Há **62 anos**
reescrevendo histórias!

O que é JSON ?

No Android, esses dados podem ser convertidos em objetos Kotlin.

Por exemplo:

```
data class Aluno(  
    val id: Int,  
    val nome: String,  
    val curso: String,  
    val idade: Int  
)
```



Há **62 anos**
reescrevendo histórias!

Arquitetura Retrofit

vamos utilizar o Retrofit, bastante utilizado no desenvolvimento Android.

Arquitetura:

Android



ViewModel



Repository



Retrofit



API REST



Servidor

Essa organização também ajuda a aplicar princípios de arquitetura como MVVM.



Há **62 anos**
reescrevendo histórias!

Arquitetura MVVM

A arquitetura **MVVM** (Model-View-ViewModel) é um padrão de projeto de software que separa a interface do usuário da lógica de negócios para facilitar testes e manutenção

Hora de Praticar



Há **62 anos**
reescrevendo histórias!

Retrofit

Biblioteca *type-safe* que converte APIs REST em interfaces Java/Kotlin. Facilita a serialização/desserialização de JSON para objetos (POJOs/Data Classes) e possui suporte nativo a Coroutines, otimizando o desenvolvimento.



Há **62 anos**
reescrevendo histórias!

OkHttp

O *motor* de rede subjacente ao Retrofit. Gerencia eficientemente conexões, *pooling*, compressão GZIP e mecanismos de cache. Permite a implementação de *interceptors* para controle e segurança das requisições.



Há **62 anos**
reescrevendo histórias!

Integração do Retrofit

1

Dependências

Adicionar as bibliotecas ``retrofit2`` e um conversor JSON (ex: ``converter-gson``) ao arquivo ``build.gradle`` do módulo para incluir as funcionalidades necessárias.

2

Definir Interface API

Criar uma interface Kotlin anotada com os métodos HTTP (ex: ``@GET("users/id")``) que mapeiam os *endpoints* da API remota.



Há **62 anos**
reescrevendo histórias!

Integração do Retrofit

3

Configurar Cliente Retrofit

Instanciar o objeto Retrofit, definindo a `baseUrl` da API e o `ConverterFactory` (ex: `GsonConverterFactory.create()`) para serialização/desserialização de dados.



Há **62 anos**
reescrevendo histórias!

Integração do Retrofit

4

Realizar Chamada Assíncrona

Executar as chamadas à API utilizando *Suspend Functions* (Kotlin Coroutines) para garantir que as operações de rede sejam realizadas em *threads* separadas, evitando bloqueios da **Main Thread**.



Há **62 anos**
reescrevendo histórias!

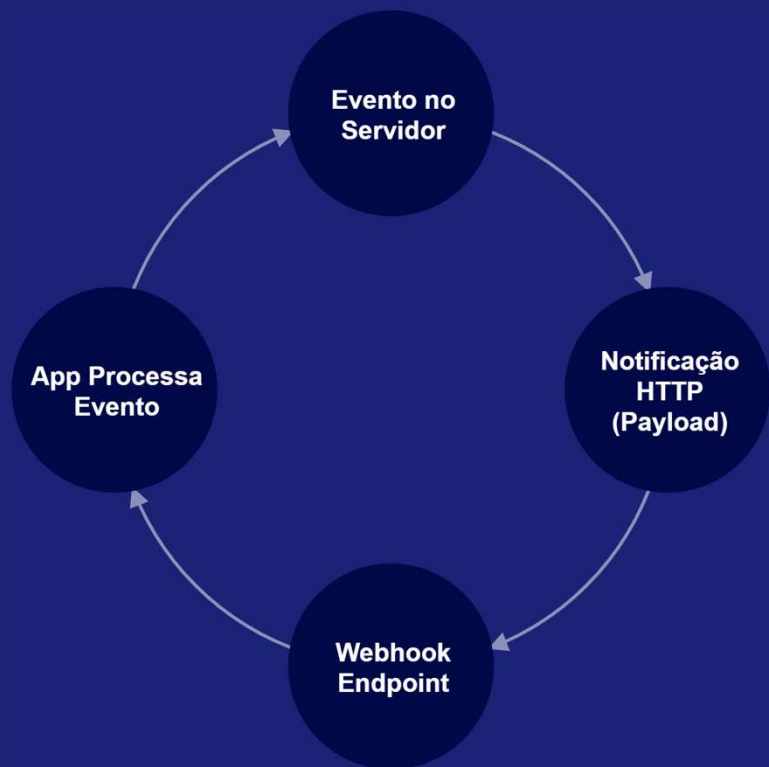
Integração do Retrofit

5

Tratar Erros

Implementar blocos **try/catch** ou utilizar o *wrapper* `Result` para gerenciar exceções de rede e respostas de erro da API de forma robusta e controlada.

Conceito de Webhooks



Diferente das APIs tradicionais, onde o aplicativo solicita dados ao servidor (modelo *Pull*), nos Webhooks, o servidor avisa o aplicativo, invertendo o fluxo de comunicação para um modelo **Push**. Quando um evento ocorre no servidor (ex: um pagamento aprovado), ele envia uma notificação HTTP para um *endpoint* pré-configurado. Isso elimina o desperdício de recursos associado ao *polling* (consultas repetitivas), promovendo maior eficiência. Webhooks são comumente utilizados em gateways de pagamento (como Stripe e Cielo), sistemas de CI/CD (ex: GitHub Actions) e APIs de mensageria (ex: WhatsApp Business API) para integrações em tempo real.



Há **62 anos**
reescrevendo histórias!

Polling (Pull)

O cliente Android realiza requisições em intervalos fixos (ex: a cada 30 segundos) para verificar novas informações. Isso resulta em **alto consumo de bateria** e latência desnecessária, pois a conexão é estabelecida mesmo sem novos dados.



Há **62 anos**
reescrevendo histórias!

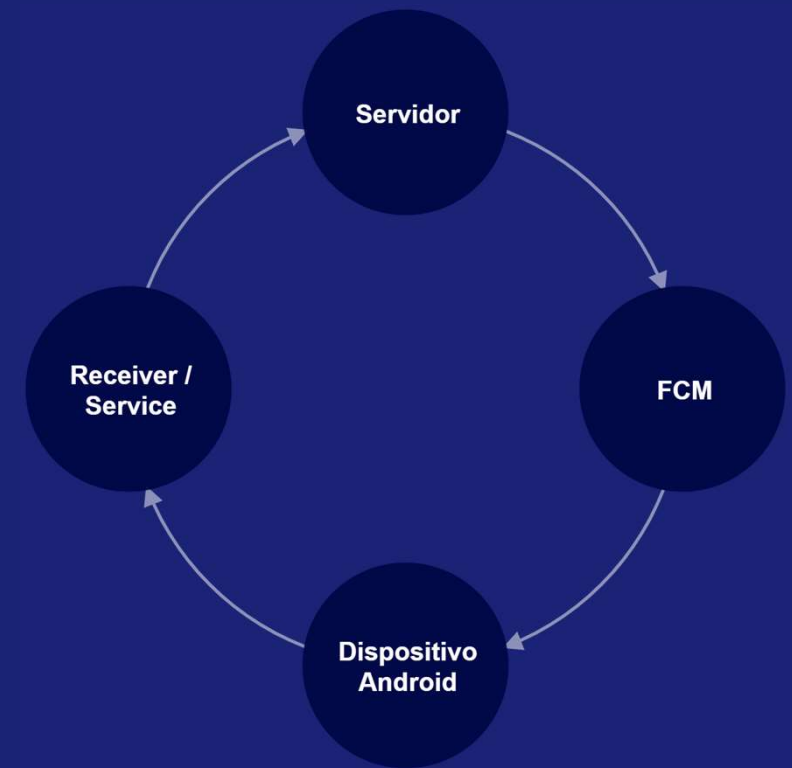
Webhooks (Push)

O servidor envia um POST para o cliente (ou serviço de Push) assim que o dado é alterado. Proporciona **reação imediata** e economia de bateria, ativando a rádio do dispositivo apenas quando há uma atualização real.

Implementação de Webhooks em Mobile

FCM como Intermediário

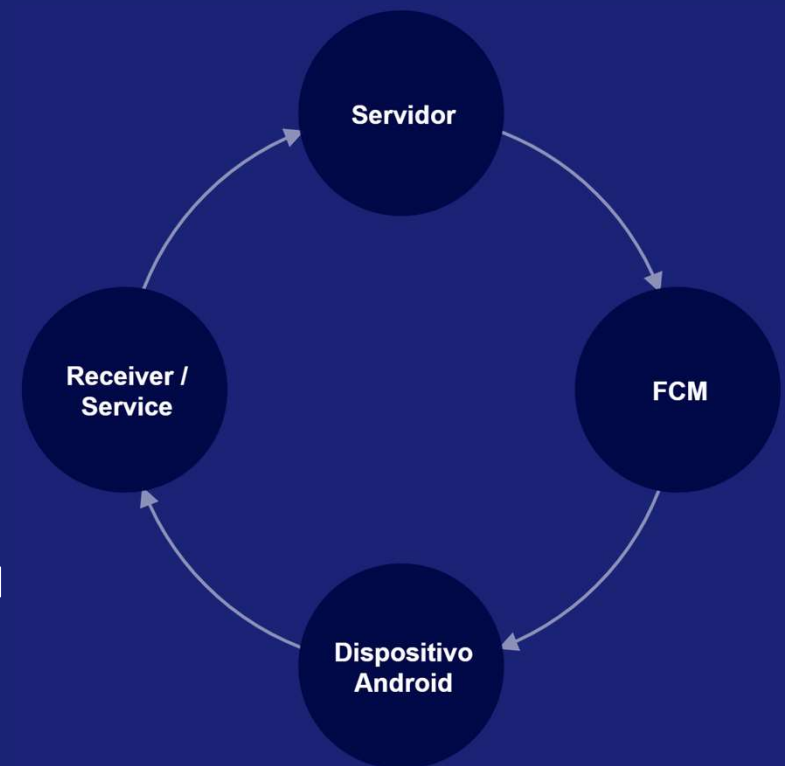
Como dispositivos móveis não mantêm IPs estáticos, o servidor envia o Webhook ao Firebase Cloud Messaging (FCM). O FCM entrega uma *Silent Notification* ao aplicativo Android, que pode receber o evento sem exibir uma notificação ao usuário.



Implementação de Webhooks em Mobile

Processamento em Background

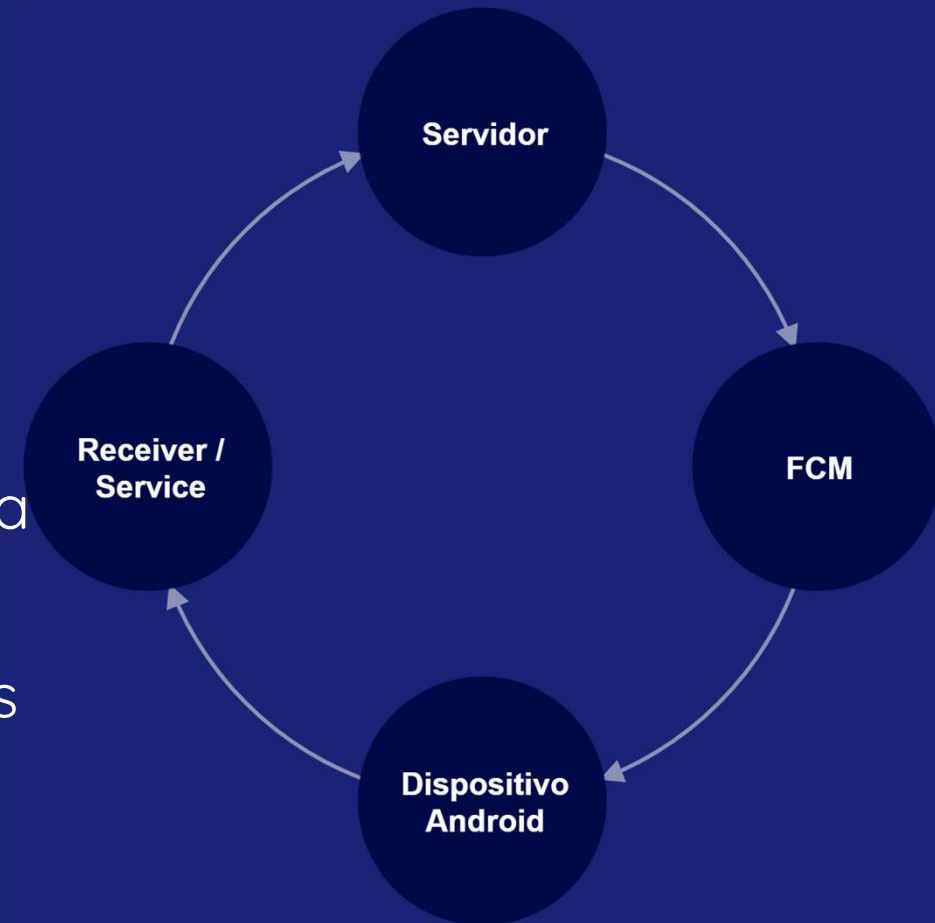
O Android recebe a mensagem por meio de um *Broadcast Receiver* e aciona um *Service* para processar os dados recebidos via Push. Esse fluxo desacopla o endpoint do servidor do dispositivo móvel e contorna a ausência de acesso direto



Implementação de Webhooks em Mobile

Segurança e Validação

Antes de processar o evento, valide a assinatura HMAC do Webhook e verifique a integridade do payload. Essa etapa confirma a origem legítima da requisição e reduz o risco de comandos forjados por um atacante.





Exercício Resolvido

Como podemos realizar uma chamada de API para buscar dados de um usuário específico utilizando Retrofit e Coroutines em uma aplicação Android?



Exercício Resolvido

1

Defina o modelo de dados (data class) que representará o usuário, incluindo seus campos e tipos correspondentes. Exemplo: ``data class User(val id: Int, val name: String)``.

2

Crie uma interface de serviço para a API (ApiService) onde você declarará o método assíncrono para buscar o usuário. Utilize anotações do Retrofit para mapear a requisição HTTP. Exemplo: ``interface ApiService @GET("users/id") suspend fun getUser(@Path("id") id: Int): Response ``.

3

Implemente a chamada da API dentro do seu ViewModel, utilizando um ``viewModelScope.launch`` para executar a Coroutine. Obtenha a resposta do repositório e atualize o LiveData (``_user.value``) com os dados do corpo da resposta se a chamada for bem-sucedida. Exemplo: ``viewModelScope.launch val response = repository.getUser(1) if(response.isSuccessful) _user.value = response.body() ``.

Qual das alternativas descreve corretamente a principal diferença entre *Polling* e *Webhooks* na comunicação entre sistemas?

A Webhooks só podem ser usados em redes locais, enquanto Polling funciona na internet.

B O Retrofit é a única biblioteca que permite implementar Webhooks em Android.

C Polling é assíncrono e Webhooks são síncronos.

D No Polling o cliente solicita atualizações; no Webhook o servidor envia atualizações quando eventos ocorrem.

Qual das alternativas descreve corretamente a principal diferença entre *Polling* e *Webhooks* na comunicação entre sistemas?



Webhooks só podem ser usados em redes locais, enquanto Polling funciona na internet.



O Retrofit é a única biblioteca que permite implementar Webhooks em Android.



Polling é assíncrono e Webhooks são síncronos.



No Polling o cliente solicita atualizações; no Webhook o servidor envia atualizações quando eventos ocorrem.

Segurança e Melhores Práticas



Segurança digital: proteja dados com HTTPS e Network Security Configuration.

- Priorize **HTTPS Always**: Nunca trafegue dados em texto claro; utilize protocolos TLS/SSL para proteger a comunicação entre o aplicativo e o servidor.
- Implemente **Network Security Configuration**: Configure um arquivo XML no Android para restringir domínios permitidos e aplicar o *Certificate Pinning*, aumentando a segurança contra ataques *man-in-the-middle*.
- Utilize um **Interceptor de Autenticação**: No OkHttp, use um `Interceptor` para injetar automaticamente tokens JWT (JSON Web Tokens) ou chaves de API em todos os cabeçalhos das requisições, simplificando a gestão de credenciais.

Segurança e Melhores Práticas

Defina uma **Timeout Policy**: Configure tempos de *timeout* adequados (Connection, Read, Write) para evitar que o aplicativo fique pendente em redes instáveis ou lentas, melhorando a experiência do usuário.



Segurança digital: proteja dados com HTTPS e Network Security Configuration.

Em um aplicativo de rastreamento de entregas em tempo real, qual seria o impacto do Polling (a cada 2 segundos) versus Webhooks (via FCM) na bateria e na experiência do usuário?

como reduzir a perda e a desordem das mensagens em uma rede móvel instável.



Conclusão

- APIs RESTful com Retrofit e OkHttp são padrão para requisições Pull, utilizando JSON e coroutines para eficiência.
- Webhooks, via modelos Push, eliminam polling e latência, usando FCM para entregar notificações eficientemente em mobile.
- A implementação em Android exige tratamento assíncrono, segurança HTTPS, validação de assinatura e interceptors de autenticação.
- Entender Polling vs. Webhooks permite escolher a estratégia ideal, otimizando bateria e experiência do usuário em aplicações.